Closure properties of Regular Languages

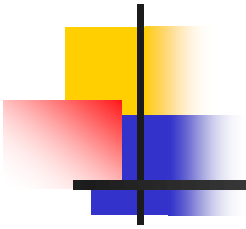


Closure properties for Regular Languages (RL)

This is different from Kleene closure

- *Closure property:*
  - If a set of regular languages are combined using an operator, then the resulting language is also regular
- Regular languages are *closed* under:
  - Union, intersection, complement, difference
  - Reversal
  - Kleene closure
  - Concatenation
  - Homomorphism
  - Inverse homomorphism

Now, lets prove all of this!



RLs are closed under union

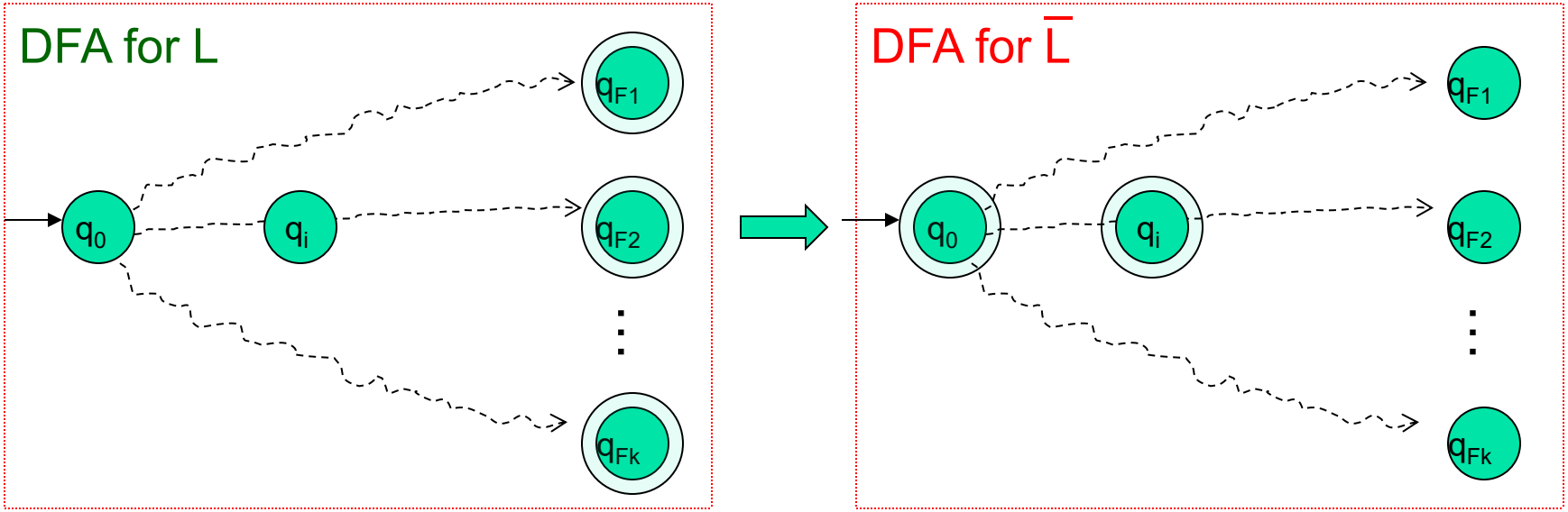
- IF L and M are two RLs THEN:
  - they both have two corresponding regular expressions, R and S respectively
  - (L U M) can be represented using the regular expression R+S
  - Therefore, (L U M) is also regular ▭

How can this be proved using FAs?

RLs are closed under complementation

- If L is an RL over ∑, then $\overline{L}$=∑*-L
  - To show $\overline{L}$ is also regular, make the following construction

Convert every final state into non-final, and every non-final state into a final state



Assumes q0 is a non-final state. If not, do the opposite.

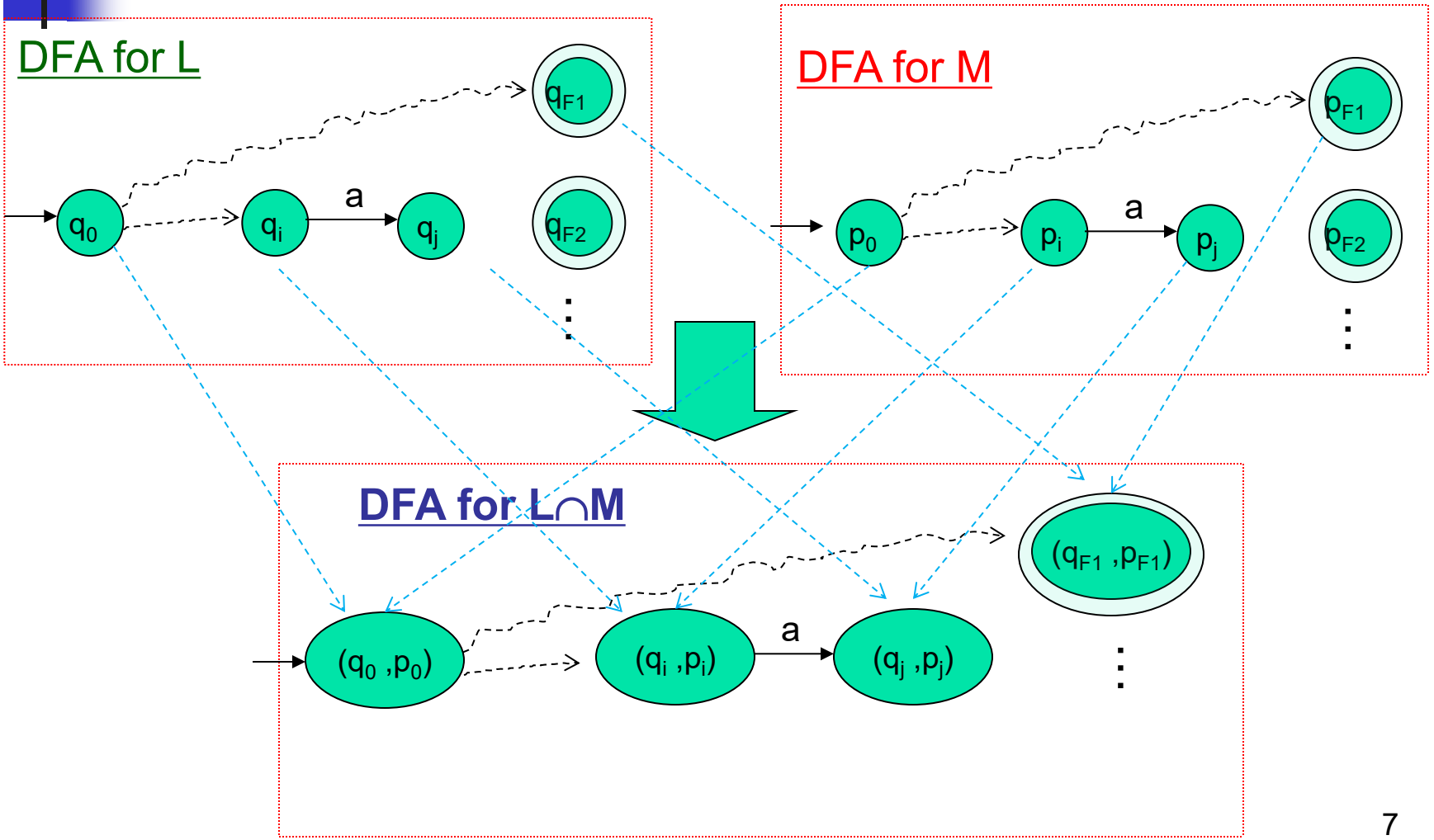
RLs are closed under intersection

- A quick, indirect way to prove:
  - By DeMorgan's law:
  - $L \cap M = (\overline{\overline{L} \cup \overline{M}})$
  - Since we know RLs are closed under union and complementation, they are also closed under intersection
- A more direct way would be construct a finite automaton for $L \cap M$

DFA construction for L ∩ M

- A_L = DFA for L = $\{Q_L, \Sigma, q_L, F_L, \delta_L\}$
- A_M = DFA for M = $\{Q_M, \Sigma, q_M, F_M, \delta_M\}$
- Build $A_{L \cap M} = \{Q_L \times Q_M, \Sigma, (q_L, q_M), F_L \times F_M, \delta\}$ such that:
  - $\delta((p,q),a) = (\delta_L(p,a), \delta_M(q,a))$, where p in Q_L, and q in Q_M
- This construction ensures that a string w will be accepted if and only if w reaches an accepting state in <u>both</u> input DFAs.

DFA construction for L ∩ M

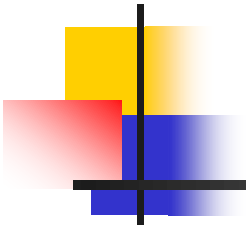


RLs are closed under set difference

- We observe:
  - $L - M = L \cap \overline{M}$
- Therefore, L - M is also regular

Closed under intersection

Closed under complementation



RLs are closed under reversal

Reversal of a string w is denoted by w^R

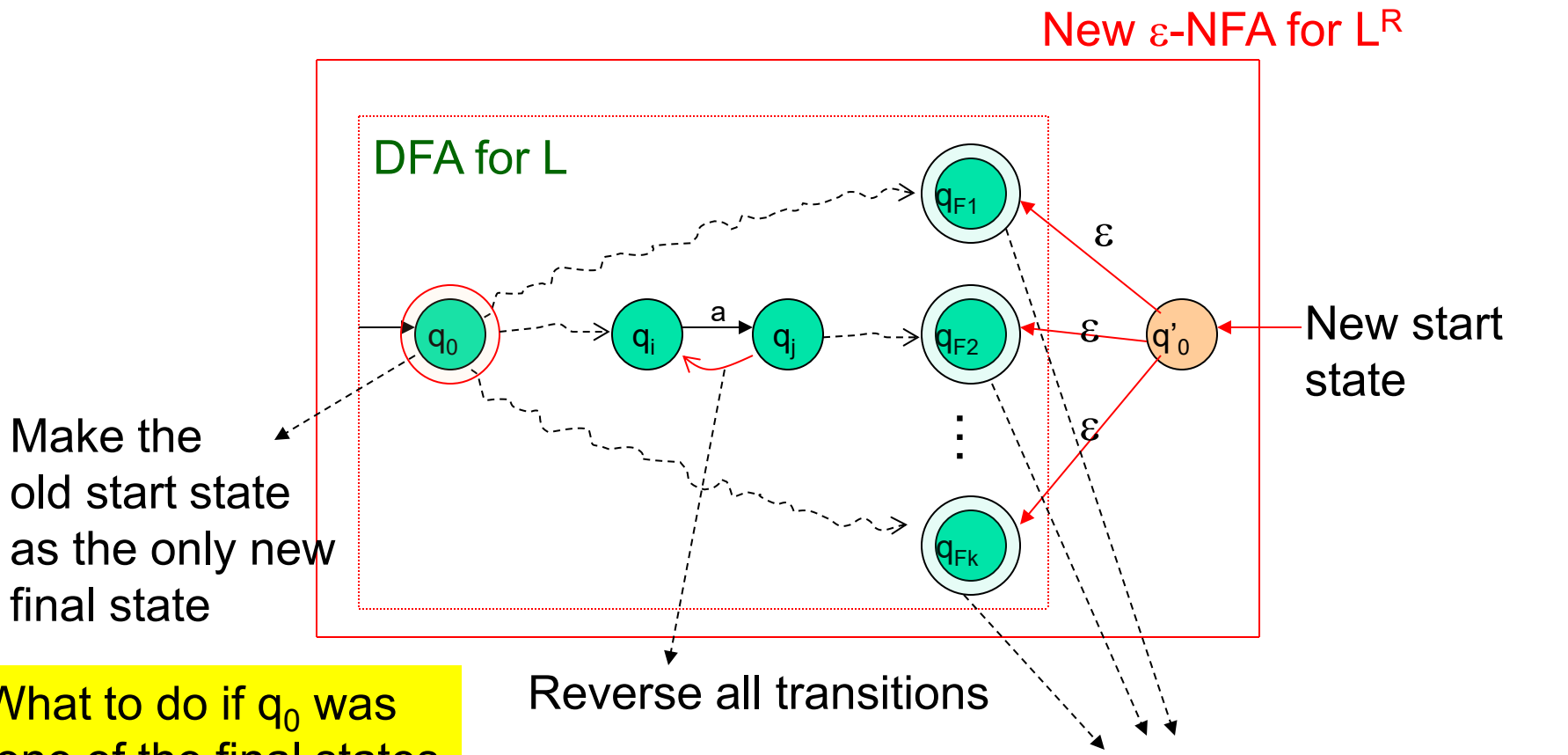
- E.g., w=00111, w^R=11100

Reversal of a language:

- L^R = The language generated by reversing all strings in L

Theorem: If L is regular then L^R is also regular

ε -NFA Construction for L^R



What to do if q_0 was one of the final states in the input DFA?

If L is regular, L^R is regular (proof using regular expressions)

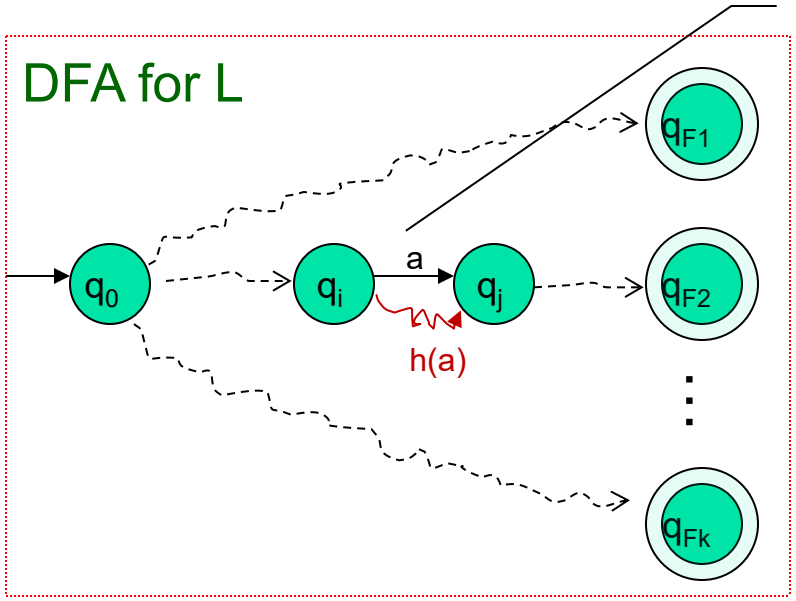
- Let E be a regular expression for L
- Given E, how to build E^R?
- Basis: If E= ε, Ø, or a, then E^R=E
- Induction: Every part of E (refer to the part as "F") can be in only *one* of the three following forms:
  1. $F = F_1+F_2$
     - $F^R = F_1^R+F_2^R$
  2. $F = F_1F_2$
     - $F^R = F_2^RF_1^R$
  3. $F = (F_1)^*$
     - $(F^R)^* = (F_1^R)^*$

Homomorphisms

- Substitute each symbol in ∑ (main alphabet) by a corresponding string in T (another alphabet)
  - h: ∑--->T*
- Example:
  - Let ∑={0,1} and T={a,b}
  - Let a homomorphic function h on ∑ be:
    - h(0)=ab, h(1)=ε
  - If w=10110, then h(w) = εabεεab = abab
- In general,
  - $h(w) = h(a_1)\, h(a_2) \ldots h(a_n)$

Given a DFA for L, how to convert it into an FA for h(L)?

FA Construction for h(L)



Replace every edge "a" by a path labeled h(a) in the new DFA

- Build a new FA that simulates h(a) for every symbol a transition in the above DFA
- The resulting FA may or may not be a DFA, but will be a FA for h(L)

Given a DFA for M, how to convert it into an FA for $h^{-1}(M)$?

The set of strings in $\sum^*$ whose homomorphic translation results in the strings of M

Inverse homomorphism

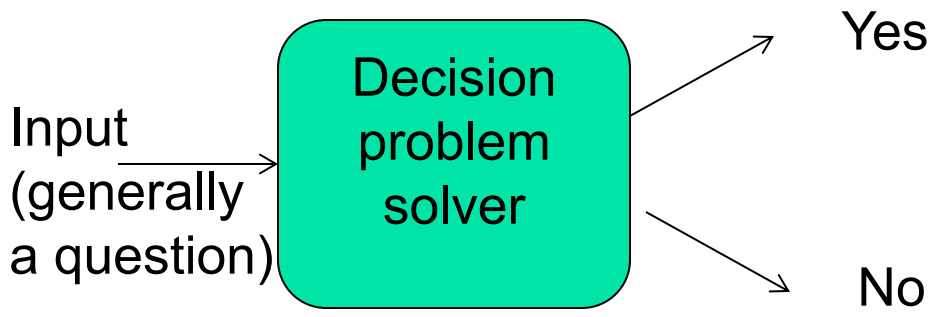
- Let h: $\sum$--->T*
- Let M be a language over alphabet T
- $h^{-1}(M) = \{w \mid w \in \sum^* \text{ s.t., } h(w) \in M\}$

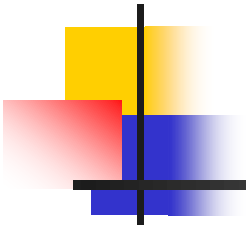
Claim: If M is regular, then so is $h^{-1}(M)$

- Proof:
  - Let A be a DFA for M
  - Construct another DFA A' which encodes $h^{-1}(M)$
  - A' is an exact replica of A, except that its transition functions are s.t. for any input symbol *a* in $\sum$, A' will simulate *h(a)* in A.
    - $\delta(p,a) = \hat{\delta}(p,h(a))$

Decision properties of regular languages

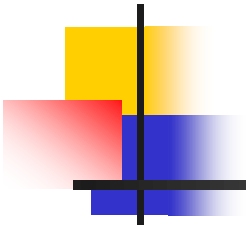
Any “decision problem” looks like this:





Membership question

- Decision Problem: Given L, is w in L?
- Possible answers: Yes or No
- Approach:
  1. Build a DFA for L
  2. Input w to the DFA
  3. If the DFA ends in an accepting state, then yes; otherwise no.



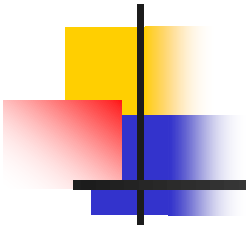
Emptiness test

- Decision Problem: Is L=Ø ?
- Approach:

  On a DFA for L:

  1. From the start state, run a *reachability* test, which returns:
     1. success: if there is at least one final state that is reachable from the start state
     2. failure: otherwise
  2. L=Ø if and only if the reachability test fails

How to implement the reachability test?



Finiteness

- Decision Problem: Is L finite or infinite?
- Approach:

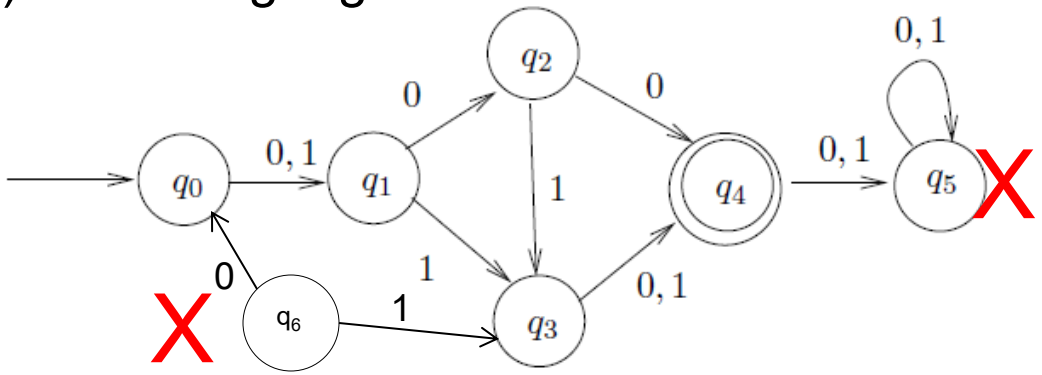
  On a DFA for L:

  1. Remove all states unreachable from the start state
  2. Remove all states that cannot lead to any accepting state.
  3. After removal, check for cycles in the resulting FA
  4. L is finite if there are no cycles; otherwise it is infinite
- Another approach
  - Build a regular expression and look for Kleene closure

How to implement steps 2 and 3?

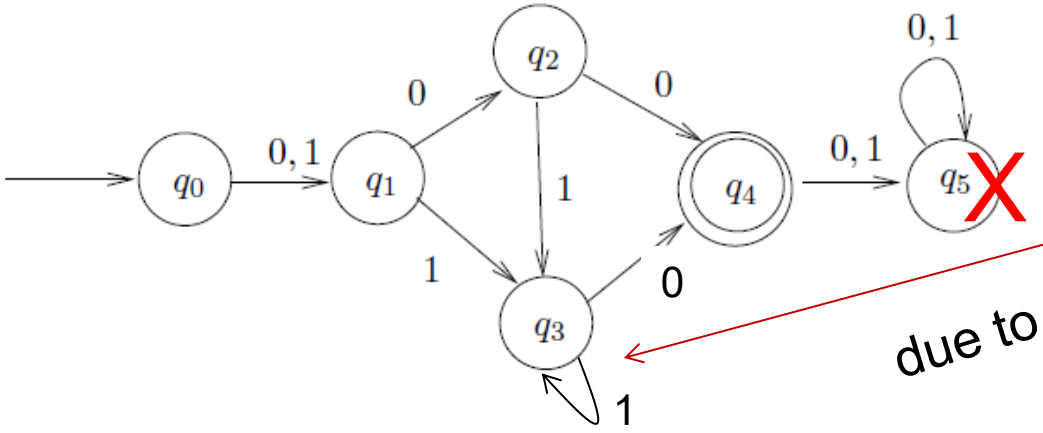
Finiteness test - examples

Ex 1) Is the language of this DFA finite or infinite?



FINITE

Ex 2) Is the language of this DFA finite or infinite?



INFINITE

due to this